OBJECT ORIENTED PROGRAMMING
CCS20704, DCS20504, CCS23803

Received Date : 3 September 2026
Submission Due Date: 23 October 2026
Lecturer : Dr. Jamal Abdullahi
Weightage : 15%
Semester : SEPTEMBER 2026

Instruction to students:

- This is a GROUP assignment.
- Complete this cover sheet and attach it to your assignment (first page).

Student declaration:	
I declare that: • ***This assignment is my/our own work.*** • ***I/we understand what is meant by plagiarism.*** • ***My lecturer has the right to deduct my marks in case of:*** - ***Late submission*** - ***Any plagiarism found in my assignment.***	
NAME: LAALY HAASAN **MATRICS NO: 012026020481** 	**NAME: SHARVINA A/P THANIKASALAM** **MATRICS NO: 012026020199** 

1.0 INTRODUCTION

Object-Oriented Programming (OOP) is a programming approach that organizes a program using classes and objects. It allows programmers to represent real-world entities by defining their attributes and methods. Attributes describe the properties of an object, while methods define the actions that the object can perform.

This assignment develops a Phone and Car Information System using Java. The system consists of three classes: Phone, Car, and Main. The Phone class stores information about a mobile phone, while the Car class stores information about a car. The Main class creates objects from these classes and demonstrates how their attributes and methods are used.

The system applies fundamental OOP concepts, including classes, objects, encapsulation, constructors, getters, setters, and methods. It also demonstrates selection structures, such as if-else and switch, and looping structures, such as for and while.

The purpose of this assignment is to develop an organized Java program that represents real-world objects, protects their data through encapsulation, and performs meaningful operations based on their attributes.

2.0 SYSTEM DESIGN

2.1 Explanation of Car Java and Phone Java

a. Class Declaration and Attributes

```java
public class Phone {

    private String brand;
    private String model;
    private int storage;
    private double price;
    private int battery;
    private String color;
    private String owner;
```

```java
ar.java
public class Car {
    private String brand;
    private String model;
    private int year;
    private double price;
    private int mileage;
    private String color;
    private String owner;
```

The coding above shows the attributes such as brand, model, storage and in both the phone.java and car.java and its respective data types such as int, double and string. The attributes are declared using the private accessor to prevent direct access from other classes. This supports encapsulation by requiring other classes to use the public methods provided by the Phone class and car class.

b. Default Constructor

```java
public Phone() {
    brand = "Unknown";
    model = "Unknown";
    storage = 0;
    price = 0.0;
    battery = 0;
    color = "Unknown";
    owner = "Unknown";
}
```

```java
public Car() {
    brand = "Unknown";
    model = "Unknown";
    year = 0;
    price = 0.0;
    mileage = 0;
    color = "Unknown";
    owner = "Unknown";
}
```

This is a default constructer because it does not have any parameters and the attributes is initialized with default values such as brand and model is initialized as unknown and numerical attributes are initialized as zero.

c. Parameterized Constructer

```
public Phone(String brand, String model, int storage, double price, int battery, String color, String owner) {
    this.brand = brand;
    this.model = model;
    this.storage = storage;
    this.price = price;
    this.battery = battery;
    this.color = color;
    this.owner = owner;
}
```

```
public Car(String brand, String model, int year,
    this.brand = brand;
    this.model = model;
    this.year = year;
    this.price = price;
    this.mileage = mileage;
    this.color = color;
    this.owner = owner;
}
```

This constructor accepts seven values when creating the phone and car object. Instead of having default values, the object will have values supplied to constructors. For an example, "this.brand = brand" means store the parameter value in the object's brand attributes and the same applies to the rest.

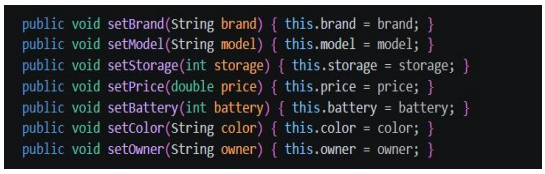
d. Gatter Methods

```
public String getBrand() { return brand; }
public String getModel() { return model; }
public int getStorage() { return storage; }
public double getPrice() { return price; }
public int getBattery() { return battery; }
public String getColor() { return color; }
public String getOwner() { return owner; }
```

```
public String getBrand() { return brand; }
public String getModel() { return model; }
public int getYear() { return year; }
public double getPrice() { return price; }
public int getMileage() { return mileage; }
public String getColor() { return color; }
public String getOwner() { return owner; }
```

A getter method is used to retrieve the value of the attributes. For an example, "System.out.println(p1.getBrand())" will retrieve the phones brand and prints "Apple ".The return method is used here to send the value back to the codes and the return type tells Java what kind of value the method returns. For example, "getBrand()" returns a String, while "getPrice()' returns a double.

e. Setter Methods



```
public void setBrand(String brand) { this.brand = brand; }
public void setModel(String model) { this.model = model; }
public void setStorage(int storage) { this.storage = storage; }
public void setPrice(double price) { this.price = price; }
public void setBattery(int battery) { this.battery = battery; }
public void setColor(String color) { this.color = color; }
public void setOwner(String owner) { this.owner = owner; }
```



```
public void setBrand(String brand) { this.brand = brand; }
public void setModel(String model) { this.model = model; }
public void setYear(int year) { this.year = year; }
public void setPrice(double price) { this.price = price; }
public void setMileage(int mileage) { this.mileage = mileage; }
public void setColor(String color) { this.color = color; }
public void setOwner(String owner) { this.owner = owner; }
```

A setter method is used to change a attributes value. For an example, "p1.setBrand("Samsung")" changes the phone brand to Samsung. Then the keyword void means the method does not return a value.

f. The display Method



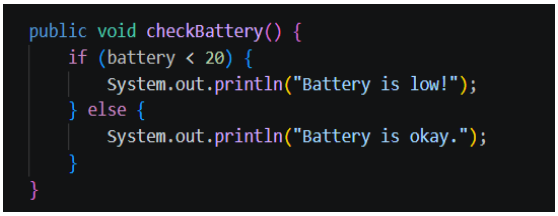
```
public void display() {
    System.out.println("Brand: " + brand);
    System.out.println("Model: " + model);
    System.out.println("Storage: " + storage + "GB");
    System.out.println("Price: " + price);
    System.out.println("Battery: " + battery + "%");
    System.out.println("Color: " + color);
    System.out.println("Owner: " + owner);
}
```



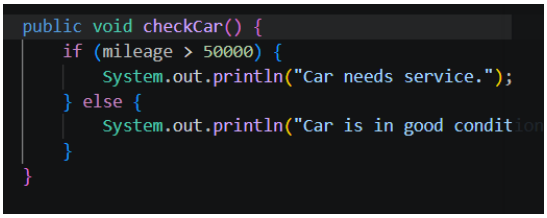
```
public void display() {
    System.out.println("Brand: " + brand);
    System.out.println("Model: " + model);
    System.out.println("Year: " + year);
    System.out.println("Price: " + price);
    System.out.println("Mileage: " + mileage);
    System.out.println("Color: " + color);
    System.out.println("Owner: " + owner);
}
```

This methos prints all the car and phone information. The "System.out.println()" displays text on the screen and the "+" joints the attributes with the value and "GB" and "%" add units to the output.

g. The checkBattery() and checkCar() method



```
public void checkBattery() {
    if (battery < 20) {
        System.out.println("Battery is low!");
    } else {
        System.out.println("Battery is okay.");
    }
}
```

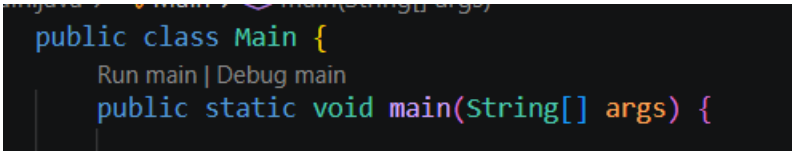


```
public void checkCar() {
    if (mileage > 50000) {
        System.out.println("Car needs service.");
    } else {
        System.out.println("Car is in good condit
    }
}
```

This uses and if-else method. For an example ,"if (battery < 20)" checks whether the battery is below 20%.If the condition is true then it prints "Battery is low!" if the statement is false it prints "Battery is okay."

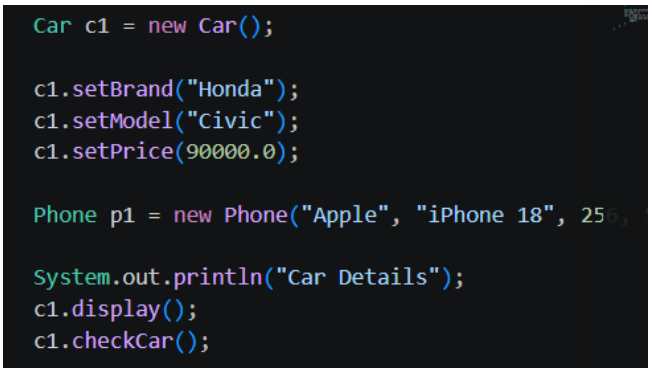
2.2 Explanation For Main Java

a. The Main Method



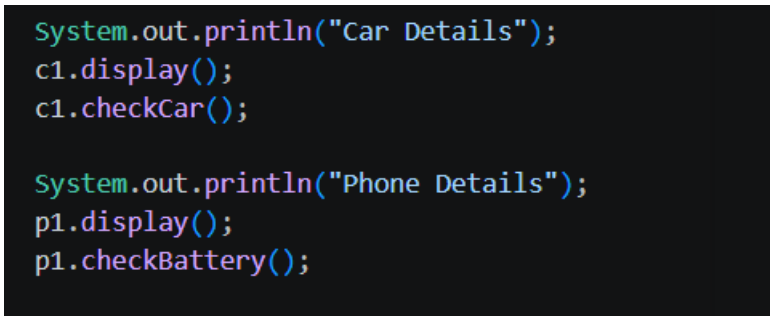
This declares the main class and is the entry point where java starts executing the program.

b. Creating The Car and Phone Object



This creates the car and phone object using the c1 and p1 using the default constructor. Moreover, the statement "c1.setBrand("Honda")", c1.setModel("Civic"), c1.setPrice(90000.0)" statement is to modify the attributes using the setter methods meanwhile other attributes remain the same.

c. Displaying the car and Phone details



First, the program prints the heading "Car Details". Then "c1.display()" prints the car information, and "c1.checkCar()" checks the car's mileage.

d. Comparing prices using if-else

```
System.out.println("If-Else Test");
if (c1.getPrice() > p1.getPrice()) {
    System.out.println("Car is more expensive.");
} else {
    System.out.println("Phone is more expensive.");
}
```

This is another example of encapsulation. The program retrieves the prices through getters rather than accessing the private fields directly and uses an if else statement.For an example, the value of car is RM 90,000.00 and phone value is RM 5 499.00. Since the car value is greater than the phone value the program prints the car is more expensive.

e. Checking the car colour using switch

```
System.out.println("Switch Test");
String col = c1.getColor();
switch (col) {
    case "White":
        System.out.println("Color is White.");
        break;
    case "Black":
        System.out.println("Color is Black.");
        break;
    default:
        System.out.println("Other color: " + col);
        break;
}
```

First, “c1.getColor()” retrieves the car's colour and stores it in col. Then the switch statement checks the value against different cases. For an example "White" prints colour is White.

f. The for loop

```
System.out.println("For Loop Test");
for (int i = 1; i <= 3; i++) {
    System.out.println("Count number: " + i);
}
```

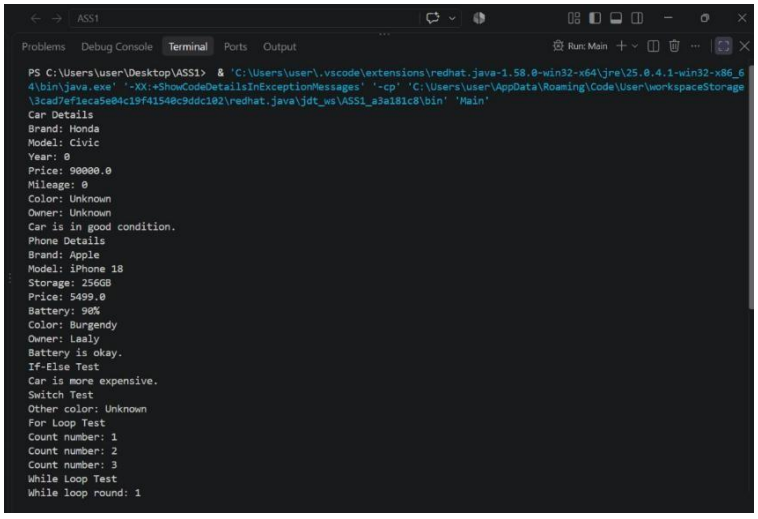
The for loop repeats a statement while a condition remains true. The loop contains three main parts. The initialization int i = 1 sets the starting value. The condition i <= 3 determines whether the loop continues, and i++ increases the counter by one after each iteration.The loop prints the numbers 1, 2, and 3. When i becomes 4, the condition is false, and the loop terminates.

g. The While Loop

```
System.out.println("While Loop Test");
int num = 1;
while (num <= 2) {
    System.out.println("While loop round: " + num);
    num++;
}
```

The while loop repeats a statement as long as its condition is true. The variable num begins at 1. The program checks whether num <= 2. If the condition is true, it prints the current value and increases num by one. The loop prints the values 1 and 2. When num becomes 3, the condition is false, and the loop terminates.

3.0 OUTPUT



4.0 CONCLUSION

In conclusion, the Phone and Car Information System demonstrates fundamental Object-Oriented Programming concepts using Java. The Phone and Car classes each contain seven private attributes, a default constructor, a parameterized constructor, seven getter methods, seven setter methods, and additional methods that perform meaningful operations.

Encapsulation is demonstrated through private attributes and public getter and setter methods. Constructors provide different ways to initialize objects, while methods display object information and check the phone's battery level or the car's milage.

The Main class demonstrates object creation, attribute manipulation, selection structures, and looping structures. These features show how Java can be used to represent real-world objects in an organized and reusable manner.

Testing the program is necessary to confirm that the implementation produces the expected results. Overall, this assignment provides practical experience in designing classes, creating objects, applying encapsulation, and implementing basic program logic in Java.

Github link: https://github.com/alhassanlaaly/OOP-Ass1

5.0 REFERENCES

Oracle. (n.d.-a). ***Control flow statements*****. The Java Tutorials.**
https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html

Oracle. (n.d.-b). ***Lesson: Classes and objects*****. The Java Tutorials.**
https://docs.oracle.com/javase/tutorial/java/javaOO/

Oracle. (n.d.-c). ***Lesson: Object-oriented programming concepts*****. The Java Tutorials.**
https://docs.oracle.com/javase/tutorial/java/concepts/